Systems Programming Knowledge

수식	변수 의미	조건	챕터
`pnb = pending & ~blocked`	pending: 전송된 시그널 비트벡터, blocked: 차단 비트벡터	pnb ≠ 0이면 처리	Ch10
$O_k = \max(P_i, i \le k) / H_k$	P_i: k번째 요청 시점의 페이로드 합계, H_k: k번째 요청 후 힙 크기	$0 < O_k \le 1$, 1에 가까울수록 효율적	Ch7
`D(Rb, Ri, S)` → `Mem[Reg[Rb] + S×Reg[Ri] + D]`	Rb: 베이스 레지스터, Ri: 인덱스(%rsp 불가), S: 스케일(1/2/4/8), D: 변위(상수)	S ∈ {1,2,4,8}	Ch2
`VA = VPN|VPO` → `PA = PPN|PPO`	VPN: 가상 페이지 번호, VPO=PPO: 페이지 오프셋(하위 p비트)	PPO = VPO (동일)	Ch6
단일 PT 크기 = $2^{n-p} \times 2^3$ bytes	n: 가상 주소 비트 수, p: 페이지 크기 비트 수	n=48, p=12 → 2^{39} = 512GB	Ch6
`pushq` = `%rsp -= 8` → store	%rsp: 스택 포인터	스택은 낮은 주소 방향으로 성장	Ch2
`popq` = load → `%rsp += 8`	%rsp: 스택 포인터		Ch2
Timing 공격 복잡도: 52^9 → 52×9	52: 알파벳 대소문자 수, 9: 비밀번호 길이	strcmp 조기 반환 이용	Ch15
Rowhammer: `VA 하위 12bit = PA 하위 12bit`	4KB 페이지 내 오프셋은 VA=PA	인접 Row 주소 추론에 활용	Ch15

예외(Exception) 분류 4가지 ★

종류	발생 원인	동기/비동기	의도성	복귀 위치	예시
Interrupt	외부 장치	비동기	-	I_next	타이머, Ctrl-C, NIC
Trap	명령어 실행	동기	의도적	I_next	syscall, gdb breakpoint
Fault	명령어 실행	동기	비의도	I_current or Abort	Page Fault, Protection Fault 불법 명령

Abort	명령어 실행	동기	비의도	Abort	어, Parity Error

레지스터 저장 규칙 ★

구분	레지스터	저장 책임	동작
Caller-saved	%rax, %rcx, %rdx, %rsi, %rdi, %r8~%r11	호출자(Caller)	Callee가 자유롭게 덮어씀. 필요 시 caller가 call 전에 저장
Callee-saved	%rbx, %r12~%r14, %rbp	피호출자(Callee)	사용 전 push, 반환 전 pop 필수
특수	%rsp	별도 규칙	항상 스택 top 유지

인자 전달 레지스터 순서 ★

순서	**1**	**2**	**3**	**4**	**5**	**6**	**7번째~**	**반환값**
레지스터	%rdi	%rsi	%rdx	%rcx	%r8	%r9	스택(역순 push)	%rax

프로세스 vs 스레드 ★

항목	프로세스	스레드
주소 공간	독립	공유 (힙·코드·데이터·라이브러리)
생성 비용	~20K cycles	~10K cycles
통신 방법	IPC (파이프/소켓/공유메모리)	공유 메모리 직접 접근
스택 보호	VM으로 격리	보호 없음 (포인터로 타 스레드 스택 접근 가능)
계층 구조	트리 (parent-child)	피어(peer) 풀
전용 자원	주소공간, fd, 레지스터, 스택	레지스터, TID, 스택 (보호 없음)

주요 시그널 ★

ID	이름	기본 동작	발생 원인	재정의 가능
2	SIGINT	종료	Ctrl-C	✓
9	SIGKILL	종료	강제 종료	✗
11	SIGSEGV	종료	Segmentation Fault	✓
14	SIGALRM	종료	타이머 만료	✓
17	SIGCHLD	무시	자식 프로세스 종료/정지	✓
20	SIGTSTP	정지	Ctrl-Z	✓

System Call 주요 번호 ★

번호	이름	설명
0	read	파일 읽기
1	write	파일 쓰기
2	open	파일 열기
3	close	파일 닫기
57	fork	프로세스 생성
59	execve	프로그램 실행
60	_exit	프로세스 종료
62	kill	시그널 전송

malloc 가용 블록 추적 방법 비교 ★

방법	구조	할당 시간	삽입 정책	특징
Implicit List	헤더 size로 전체 순회	O(전체 블록)	-	구현 단순, 현대 malloc엔 미사용
Explicit List	free 블록에 prev/next	O(가용 블록)	LIFO/FIFO/주소순	빠름, 최소 블록 크기↑
Segregated List	크기 클래스별 리스트	O(1) 근사	first-fit	처리량↑, 단편화↓, first-fit≈best-fit

Stack Buffer Overflow 공격-방어 ★

공격 기법	방어 기법	원리
Code Injection	NX Bit (XD bit)	스택을 non-executable로 마킹
NX 우회	ROP	기존 코드의 Gadget(ret 끝) 조합
ROP/Code Injection	ASLR	실행마다 주소 랜덤화 → 예측 불가
모든 스택 오버플로우	Stack Canary	%fs:0x28 랜덤값 삽입 → 반환 전 검사
직접	안전한 코딩	gets→fgets, strcpy→strncpy

I/O 함수 계층 선택 가이드 ★

상황	권장	이유
디스크/터미널 파일	Standard I/O (fopen, fprintf 등)	버퍼링, Short count 자동 처리
시그널 핸들러 내부	Unix I/O (read/write)	async-signal-safe 보장
네트워크 소켓	RIO	Short count 처리 + 소켓 호환

최고 성능 필요	Unix I/O	시스템 콜 직접 호출, 최소 오버헤드
Standard I/O + 소켓	✖ 사용 금지	스트림/소켓 제약 충돌

GC 알고리즘 비교

항목	Reference Counting	Mark & Sweep
해제 시점	count = 0 즉시	Stop-the-world 후 일괄
순환 참조	✖ 처리 불가	✓ 처리 가능
시간 오버헤드	중간 (count 관리)	높음/불규칙
공간 오버헤드	count 저장 공간	마크 비트 공간
사용 예	Chrome/Firefox C++	Java, Python, Go

Memory Consistency Model 비교

모델	채택	특징	허용 재정렬
Sequential Consistency	(이론)	가장 강력, 직관적	없음
TSO (Total Store Ordering)	x86	store buffer 허용	Store→Load
Weak Ordering	ARM	거의 모든 재정렬 허용	대부분

Breakpoint 3종류 비교 ★

종류	구현 방식	개수 제한	코드 수정	용도
Software	`int 3` (0xcc) 삽입 → SIGTRAP	무제한	필요 (PTRACE_POKETEXT)	실행 흐름 중단
Hardware	CPU DR0~DR3 레지스터	최대 4개	불필요	실행 흐름 중단
Memory	페이지 권한 변경 → Protection Fault	제한적	불필요	메모리 접근(읽기/쓰기) 감지

ptrace 주요 Request 명령 ★

Request	동작	사용 시점
PTRACE_TRACEME	자신을 부모가 trace하도록 허용	자식 프로세스 초기화 시
PTRACE_PEEKTEXT/DATA	Tracee 메모리 word 읽기	코드/데이터 확인
PTRACE_POKETEXT/DATA	Tracee 메모리 word 쓰기	브레이크포인트(0xcc) 삽입

PTRACE_GETREGS	Tracee 레지스터 전체 복사	중단 시 레지스터 값 읽기
PTRACE_SETREGS	Tracee 레지스터 전체 덮어쓰기	레지스터 값 조작
PTRACE_CONT	정지된 tracee 재개	브레이크포인트 후 계속 실행
PTRACE_SYSCALL	다음 syscall 진입/탈출 시 정지	strace 구현
PTRACE_SINGLESTEP	명령어 1개 실행 후 정지	stepi 구현

Security Attacks 비교 ★

공격	대상	원리	SW 버그 필요	대응책
Cold Boot	DRAM 암호화 키	냉각(-50°C)으로 데이터 소멸 지연	✕ 물리 공격	TRESOR
Timing Side-Channel	비밀 정보	실행 시간 차이 측정	✓ 구현 결함	Constant-time 구현
Cache Side-Channel	메모리 접근 패턴	Flush+Reload 시간 측정	✕ HW 특성	캐시 파티셔닝
Meltdown	커널 메모리	Speculative Exec + 캐시 채널	✕ HW 설계 결함	KPTI
Rowhammer	DRAM 비트	인접 Row 반복 접근 → 비트 플립	✕ HW 물리 결함	ECC DRAM, TRR

Meltdown vs Rowhammer ★

항목	Meltdown	Rowhammer
공격 레이어	CPU (투기적 실행)	DRAM (물리적 비트 플립)
정보 유출 방식	커널 메모리 읽기	PTE 변조 → 권한 상승
캐시 활용	Flush+Reload (covert channel)	clflush (cache bypass)
대응	KPTI (SW 패치, 5~20% 저하)	ECC DRAM (HW 교체)

용어	정의
ASLR	Address Space Layout Randomization. 실행마다 스택/힙/라이브러리 주소 랜덤화. ROP/Code Injection의 주소 예측 방해
async-signal-safe	시그널 핸들러 내에서 안전하게 호출 가능한 함수. 재진입 가능하거나 시그널에 중단되지 않음 (write, kill, _exit ✓ / printf, malloc ✗)
Blocked (시그널)	커널이 프로세스에 전달하지 않도록 억제된 시그널 상태. sigprocmask로 설정

Breakpoint	프로그램 실행을 특정 지점에서 중단시키는 디버깅 메커니즘. Software/Hardware/Memory 3종류
brk 포인터	힙(Heap)의 끝을 가리키는 포인터. sbrk(n)으로 n바이트 증가시켜 힙 확장
Cache Line	캐시와 메모리 사이 데이터 이동의 최소 단위. x86-64에서 64bytes
Cache Side-Channel	캐시 hit/miss 접근 시간 차이를 이용해 피해자의 메모리 접근 패턴을 추론하는 공격
Callee-saved 레지스터	피호출 함수가 사용 전 저장하고 반환 전 복원해야 하는 레지스터. %rbx, %r12~%r14, %rbp
Caller-saved 레지스터	호출자가 필요 시 저장해야 하는 레지스터. Callee가 자유롭게 덮어씀. %rax, %rcx, %rdx, %rsi, %rdi, %r8~%r11
clflush	x86 캐시 라인 강제 비우기 명령어. Rowhammer의 cache bypass와 Flush+Reload 공격에 활용
Coalescing	해제된 인접 가용 블록들을 하나로 합치는 동작. 외부 단편화 방지. Immediate/Deferred 정책
Cold Boot Attack	DRAM을 냉각(-50°C)하여 데이터 소멸을 지연시킨 후 DIMM을 분리해 다른 머신에서 암호화 키를 추출하는 물리 공격
COW (Copy-on-Write)	쓰기 발생 전까지 물리 페이지를 공유. 쓰기 시 Protection Fault → 새 페이지 복사. fork() 성능 최적화의 핵심
CR3 레지스터	현재 프로세스의 L1 페이지 테이블 물리 주소를 저장. 컨텍스트 스위치 시 교체됨
Critical Section	공유 자원에 접근하는 코드 구간. 동시에 하나의 스레드만 실행되어야 함
CSE	Common Subexpression Elimination. 동일한 부분식을 한 번만 계산하는 컴파일러 최적화
Dangling Pointer	이미 해제된 메모리를 가리키는 포인터. Use-After-Free 취약점의 원인
Deadlock	둘 이상의 프로세스/스레드가 서로 상대방이 보유한 자원을 무한히 기다리는 상태
Double-Free	이미 해제된 메모리를 다시 free()하는 버그. 가용 리스트 오염 및 동일 블록 이중 반환 유발
DR0~DR3	x86-64 CPU의 하드웨어 디버그 레지스터. 최대 4개의 Hardware Breakpoint 주소 저장. DR7로 활성화 제어
dup2(oldfd, newfd)	newfd를 oldfd의 복사본으로 만드는 시스템 콜. newfd가 열려있으면 먼저 닫음. I/O 리다이렉션 구현에 사용
ECC DRAM	Error Correcting Code 내장 DRAM. 단일 비트 플립 감지·수정. Rowhammer 방어에 효과적
Exception Table	예외 번호(k)를 인덱스로 핸들러 주소를 저장하는 OS 구성 테이블. 인터럽트 벡터라고도 함
execve()	현재 프로세스에 새 프로그램 로드·실행. PID 유지, 열린 fd 유지, 메모리 교체. 성공 시 반환 없음
External Fragmentation	가용 메모리 합계는 충분하나 연속된 단일 블록이 없어 할당 실패하는 단편화
File Descriptor (fd)	열린 파일을 식별하는 작은 정수. 프로세스별 Descriptor Table에서 관리. 기본: 0(stdin),

	1(stdout), 2(stderr)
Flush+Reload	공유 메모리 캐시를 clflush로 비운 뒤 victim 접근 후 재접근 시간을 측정하는 cache side-channel 기법
fork()	부모 프로세스의 복사본인 자식 프로세스 생성. 1번 호출 2번 반환. 부모→자식PID, 자식→0
Gadget (ROP)	ret 명령어(0xc3)로 끝나는 기존 코드의 명령어 시퀀스. 여러 Gadget을 스택에 연결해 임의 동작 수행
Heap Overflow	힙 버퍼 경계를 넘어 인접 메타데이터(prev/next 포인터)를 덮어쓰는 취약점. Arbitrary Write로 이어짐
Heap Spray	힙의 특정 위치에 원하는 데이터를 대량으로 채워넣는 기법. UAF 익스플로잇에서 vtable 조작에 사용
Implicit Blocking	시그널 핸들러 실행 중 동일 타입 시그널이 자동으로 차단되는 메커니즘. 핸들러 재진입 방지
int 3 (0xcc)	x86 소프트웨어 브레이크포인트 명령어 (1바이트). 실행 시 SIGTRAP 발생. GDB가 PTRACE_POKETEXT로 삽입
Internal Fragmentation	블록 크기가 실제 페이로드보다 커서 낭비되는 내부 공간. 정렬 패딩과 헤더 오버헤드가 원인
ISA	Instruction Set Architecture. 프로세서 명령어 집합과 프로그래머에게 보이는 상태(레지스터, 메모리 등) 정의
KPTI	Kernel Page Table Isolation. Meltdown 대응책. 커널/유저 모드 페이지 테이블 완전 분리. 컨텍스트 스위치 시 5~20% 성능 저하
leaq	Load Effective Address Quadword. 메모리를 읽지 않고 주소값 자체를 목적지 레지스터에 저장. 곱셈 최적화에도 활용
Locality (지역성)	프로그램이 최근(시간적) 또는 인접(공간적) 주소를 재사용하는 경향. 캐시 계층의 효율성 근거
Mark & Sweep	GC 알고리즘. Root에서 DFS로 도달 가능한 블록 마크 후 미마크 블록을 가용 리스트에 추가. 순환 참조 처리 가능
Meltdown	CPU 투기적 실행(Speculative Execution) + 캐시 covert-channel을 결합하여 커널 메모리를 유출하는 HW 설계 결함 공격
MMU	Memory Management Unit. 가상 주소를 물리 주소로 변환하는 하드웨어 유닛. TLB 포함
mmap()	파일이나 익명 메모리를 프로세스 가상 주소 공간에 매핑하는 시스템 콜. MAP_SHARED/MAP_PRIVATE/MAP_ANON
Mutex	Mutual Exclusion. 임계 구역 상호 배제를 보장하는 동기화 도구. test_and_set 원자적 연산으로 구현
NX Bit	No-Execute. 메모리 영역의 실행 권한을 제거하여 Code Injection 방어. PTE의 XD(eXecute Disable) 비트
Open File Table	모든 프로세스가 공유하는 테이블. 각 항목에 file pos(현재 위치)와 refcnt(참조 횟수) 저장
Orphan Process	부모가 먼저 종료된 자식 프로세스. init(PID=1)이 자동으로 입양하여 reaping

Page Fault	접근한 가상 페이지가 물리 메모리에 없을 때 발생하는 Fault 예외. 커널이 디스크에서 페이지 로드 후 재실행
Page Table	VPN(Virtual Page Number)을 PPN(Physical Page Number)으로 매핑하는 커널 자료구조. 프로세스마다 독립적
Pending (시그널)	전송됐지만 아직 수신(처리)되지 않은 시그널 상태. 같은 타입은 최대 1개만 pending (큐 없음)
Process	실행 중인 프로그램의 인스턴스. Logical Control Flow + Private Address Space 두 가지 핵심 추상화 제공
Progress Graph	두 스레드의 실행 상태 공간을 2D로 표현한 그래프. 축=명령어 진행, 점=실행 상태. Race Condition 분석 도구
PTE	Page Table Entry. VPN→PPN 매핑 + 권한 비트(P, R/W, U/S, D, A, XD) 포함. Core i7은 64bit PTE
ptrace	한 프로세스(tracer)가 다른 프로세스(tracee)의 메모리/레지스터를 관찰·제어하는 시스템 콜. GDB·strace의 핵심
Race Condition	공유 자원에 대한 비원자적 접근으로 결과가 스레드 실행 순서에 따라 달라지는 버그
Reentrant Function	공유 변수(전역/정적)를 사용하지 않아 동기화 없이 여러 스레드에서 동시 호출 가능한 함수. Thread-safe의 부분집합
ROP	Return-Oriented Programming. 기존 코드의 Gadget(ret으로 끝나는 명령 시퀀스)을 조합하여 NX Bit를 우회하는 공격
Rowhammer	DRAM의 인접 Row를 반복 접근(hammer)하여 전하 누설로 victim row의 비트를 뒤집는 하드웨어 공격
Semaphore	음수가 될 수 없는 정수형 동기화 변수. wait(P: s–)와 post(V: s++) 원자적 연산으로 조작. 불변: s≥0
Short Count	read/write 시 요청한 바이트보다 적게 처리되는 정상적 현상. EOF/터미널 입력/네트워크 소켓에서 발생. 오류 아님
Side-Channel Attack	알고리즘 취약점이 아닌 구현의 부수 정보(타이밍, 전력 소비, 음향, 전자기 누출)를 이용한 공격
Signal	커널이 프로세스에게 이벤트 발생을 알리는 작은 메시지. ID 1~30. 큐 없음(같은 타입 최대 1개 pending)
Speculative Execution	CPU가 성능을 위해 분기 결과·권한 확인이 완료되기 전에 명령어를 미리 실행하는 기법. Meltdown의 근본 원인
Splitting	가용 블록을 요청 크기만큼만 잘라 할당하고 나머지를 새 가용 블록으로 분리하는 동작
Stack Canary	함수 진입 시 버퍼와 반환 주소 사이에 배치하는 랜덤값(%fs:0x28). 반환 전 값 비교하여 스택 오버플로우 탐지
Starvation	전체 시스템은 진행되나 특정 프로세스/스레드가 필요한 자원을 무한히 얻지 못하는 상태
Strength Reduction	곱셈·나눗셈 등 비용이 큰 연산을 시프트·덧셈 등 저렴한 연산으로 대체하는 컴파일러 최적화

Thread Safety	여러 스레드가 동시에 호출해도 항상 올바른 결과를 반환하는 함수의 속성. Reentrant ⊂ Thread-safe
Thrashing	프로세스의 Working Set이 물리 메모리를 초과하여 페이지 교체(swap)가 폭주하는 현상
TLB	Translation Lookaside Buffer. MMU 내부의 소형 PTE 캐시. Hit 시 메모리 접근 1회로 주소 변환 완료
Tracer / Tracee	ptrace에서 제어하는 프로세스(tracer, 예: GDB)와 제어받는 프로세스(tracee, 예: 디버깅 대상)
Transient Instruction	투기적 실행 중 실행됐다가 커밋되지 않고 되돌려지는 명령어. Meltdown 공격의 핵심 개념
TRESOR	암호화 키와 AES 연산을 CPU 레지스터/캐시에만 유지하여 Cold Boot Attack을 방어하는 Linux 커널 패치
Use-After-Free (UAF)	해제된 메모리를 재사용하는 취약점. Dangling Pointer + Heap Spray → vtable 조작 → 제어 흐름 탈취
v-node Table	파일의 실제 메타데이터(크기, 타입, 접근 권한, inode 등 stat 구조체 내용)를 저장하는 커널 테이블
Virtual Memory	물리 메모리보다 큰 주소 공간 제공 + 프로세스 격리를 구현하는 OS 추상화. 캐싱/관리/보호 3가지 역할
VPN / VPO	Virtual Page Number(상위 비트, 페이지 테이블 인덱스) / Virtual Page Offset(하위 p비트, 페이지 내 위치)
Zombie Process	종료됐지만 부모가 wait()를 호출하지 않아 exit status 등 일부 자원을 점유 중인 프로세스. ps에서 `<defunct>`

Ch1. Introduction

핵심 개념 - Reality #1: Int ≠ Integer, Float ≠ Real → 유한 비트 → 오버플로우, 결합법칙 위반 - Reality #2: 어셈블리 이해 필수 → 버그 분석, 컴파일러 최적화, 역공학 - Reality #3: 메모리 유한 자원 → C는 메모리 보호 없음 - Reality #4: 성능 ≠ 점근 복잡도 → 메모리 접근 패턴 결정적 - Reality #5: I/O·네트워크도 시스템의 일부

예시 문제

Q. `copyij`와 `copyji`의 성능 차이(19배)를 지역성 관점에서 설명하라.

```
A. C 2D 배열은 Row-Major Order로 저장됨.
   copyij: a[i][j] 접근 → 연속 메모리 순차 접근 → 공간적 지역성 ↑ → Cache Hit 多
   copyji: a[i][j] 접근 시 i가 내부 루프 → N칸씩 건너뜀 → Cache Miss 多
   → Cache Miss마다 DRAM 접근(~100 cycles) vs Cache Hit(~4 cycles) → 약 25배 비용 차이
```

Ch2-1. Machine Basics

메모리 주소 지정 일반형

```
D(Rb, Ri, S) = Mem[Reg[Rb] + S × Reg[Ri] + D]

  Rb  : Base register (16개 모두 가능)
  Ri  : Index register (%rsp 사용 불가)
  S   : Scale (1, 2, 4, 8 중 하나)
  D   : Displacement (상수, 1/2/4 bytes)
```

예시 문제

Q. `%rdx = 0xf000`, `%rcx = 0x0100` 일 때 `0x80(%rdx,%rcx,4)` 의 주소는?

```
풀이:
  Mem[ Reg[%rdx] + 4 × Reg[%rcx] + 0x80 ]
= Mem[ 0xf000 + 4 × 0x0100 + 0x80 ]
= Mem[ 0xf000 + 0x400 + 0x80 ]
= Mem[ 0xf480 ]
```

leaq vs movq 비교

```
leaq 8(%rdi), %rax   → %rax = %rdi + 8    (주소값 저장, 메모리 접근 없음)
movq 8(%rdi), %rax   → %rax = Mem[%rdi+8] (메모리에서 값 로드)
```

Ch2-2. Machine Control

Condition Code 설정 및 활용

```
cmpq %rsi, %rdi   →  %rdi - %rsi 계산 (저장 안함)
  ZF=1 if %rdi == %rsi
  SF=1 if %rdi - %rsi < 0
  CF=1 if unsigned borrow 발생
  OF=1 if signed overflow 발생

testq %rax, %rax  →  %rax & %rax (저장 안함)
  ZF=1 if %rax == 0
  SF=1 if %rax < 0   (부호 비트 확인)
```

루프 변환 예시

```
        // 원본 while
        while (x > 0) { x--; }

        // Jump-to-Middle 변환 (-Og)
            goto test;
        loop:
            x--;
        test:
            if (x > 0) goto loop;
```

Ch2-3. Procedures (스택 프레임)

스택 프레임 레이아웃

```
높은 주소
┌──────────────────────┐
│   caller의 인자 7~   │  (7개 이상 인자 시)
├──────────────────────┤
│   반환 주소          │  ← call 명령어가 push
├──────────────────────┤  ← 새 %rbp (선택적)
```

```
│   saved callee regs │  ← callee-saved 레지스터 백업
├─────────────────────┤
│   지역 변수          │
├─────────────────────┤
│   인자 빌드 공간     │  (다음 함수 호출용 7번째~ 인자)
└─────────────────────┘   ← %rsp
낮은 주소
```

예시 문제

Q. 아래 코드에서 `pcount` 재귀 호출 시 `%rbx`를 저장하는 이유는?

```c
long pcount(unsigned long x) {
    if (x == 0) return 0;
    return (x & 1) + pcount(x >> 1);
}
```

```
A. %rbx는 Callee-saved 레지스터.
   pcount는 x & 1 값을 재귀 호출 이후에도 사용해야 함.
   재귀 호출(call pcount) 시 %rbx가 덮어씌워질 수 있으므로
   진입 시 push %rbx로 저장, 반환 전 pop %rbx로 복원.
```

Ch3. Stack Buffer Overflow

스택 취약점 구조

```
함수 내 char name[32] 선언 시:

높은 주소
┌──────────────────┐
│   RET ADDR       │  ← name 기준 +40 bytes
├──────────────────┤
│   saved %rbp     │  ← name 기준 +32 bytes
├──────────────────┤ ← %rbp
│   name[0..31]    │  ← %rbp - 0x20
└──────────────────┘ ← %rsp
낮은 주소

name에 40바이트 이상 쓰면 RET ADDR 덮어씀
→ ret 실행 시 공격자가 원하는 주소로 점프
```

공격-방어 흐름

```
Code Injection
  → NX Bit (스택 non-executable) 로 방어
  → ROP 로 우회 (기존 실행 가능 코드의 Gadget 체인)
  → ASLR 로 방어 (Gadget 주소 예측 불가)
  → Stack Canary 로 추가 방어 (오버플로우 탐지)
```

예시 문제

Q. `name[32]` 버퍼가 `rbp-0x20`에 위치할 때, RET ADDR을 덮어쓰려면 몇 바이트를 입력해야 하는가?

```
풀이:
  name[] 시작 → saved %rbp: 32 bytes
  saved %rbp (8 bytes)
  RET ADDR 시작: 32 + 8 = 40 bytes 위치
  → RET ADDR을 덮어쓰려면 최소 41번째 바이트부터 4/8바이트 제어 필요
  답: 40바이트(패딩) + 8바이트(원하는 주소) = 48바이트 입력
```

Ch4. Memory Hierarchy

캐시 계층 구조

```
접근 시간    용량       위치
~0 cycles    수십 B     ┌──────────────────────┐
                        │  CPU Registers (L0)  │
~4 cycles    32 KB      ├──────────────────────┤
                        │  L1 Cache (SRAM)     │
~10 cycles   256 KB     ├──────────────────────┤
                        │  L2 Cache (SRAM)     │
~수십 cycles 8 MB      ├──────────────────────┤
                        │  L3 Cache (SRAM)     │
~100 cycles 수십 GB    ├──────────────────────┤
                        │  DRAM (Main Memory)  │
~10^7 cycles 수 TB     ├──────────────────────┤
                        │  SSD / HDD (Disk)    │
                        └──────────────────────┘
```

예시 문제

Q. 아래 두 함수 중 캐시 효율이 좋은 것은? 이유를 설명하라.

```c
// A: sum_rows
for(i=0; i<M; i++)
  for(j=0; j<N; j++) sum += a[i][j];

// B: sum_cols
for(j=0; j<N; j++)
  for(i=0; i<M; i++) sum += a[i][j];
```

```
A가 캐시 효율 좋음.
C 2D 배열은 Row-Major Order:
  a[0][0], a[0][1], ..., a[0][N-1], a[1][0], ...
A: a[i][j] → a[i][j+1] → 연속 메모리 → 공간적 지역성 ↑
B: a[0][j] → a[1][j] → N*4 bytes 건너뜀 → Cache Miss 빈번
```

Ch5. Code Optimization & Linking

컴파일러 최적화 기법 요약

```
Constant Folding:  0xFF << 8          → 0xFF00          (컴파일 시간
계산)
Strength Reduction: b * 5             → (b<<2) + b       (곱셈→시프트)
Code Motion:       루프 내 n*i        → 루프 밖으로 이동  (루프 불변)
CSE:              v[i].x * v[i].x     → t=v[i].x; t*t    (중복 계산 제
거)
Inlining:         pred(y) 호출        → 본문 직접 삽입    (함수 오버헤드
제거)
Loop Unrolling:   i++ 4번 반복        → i+=4 한 번        (분기 감소)
```

링킹 3단계

```
Step 1: 심볼 테이블 구성 (어셈블러)
  Global: non-static 함수/전역변수 → 다른 파일에서 참조 가능
  Local:  static 함수/변수          → 같은 파일 내에서만
  External Reference: 다른 파일에서 정의된 심볼 참조
```

```
Step 2: 심볼 해석 (링커)
  각 External Reference → 해당 정의(Definition)에 연결

Step 3: 재배치 (링커)
  .o 파일의 임시 주소(0x0) → 최종 실행파일의 절대 주소로 교체
  (objdump -r -d로 재배치 엔트리 확인 가능)
```

Ch6. Virtual Memory

가상 주소 → 물리 주소 변환 흐름

```
CPU가 VA 생성
     │
     ▼
┌──────────────────────────────────────┐
│  VA = VPN[1] | VPN[2] | VPN[3] | VPN[4] | VPO  │
│       9bit     9bit     9bit    9bit     12bit   │
└──────────────────────────────────────┘
     │
     ▼ TLB 조회 (VPN → PPN 캐시)
   Hit ──────────────────────────┐
   Miss                          │
     │                           │
     ▼                           │
  L1 PT (CR3 → 물리주소)         │
     │ → L2 PT → L3 PT → L4 PT  │
     │            PPN 획득       │
     └──────────────────────────┘
                                 ▼
                       PA = PPN | VPO
                                 │
                                 ▼
                       L1 Cache 조회
                       (Hit → 데이터 반환)
                       (Miss → DRAM 접근)
```

Page Fault 처리 7단계

```
① CPU가 VA 접근 → MMU가 PTE 조회
② PTE의 valid bit = 0 → Page Fault 예외 발생
③ 커널의 Page Fault Handler 실행
④ Victim 페이지 선택 (LRU 등 교체 정책)
⑤ Victim dirty 시 디스크에 write-back
⑥ 필요한 페이지를 디스크에서 물리 메모리로 로드
⑦ PTE 갱신 (valid=1, PPN 설정) → iret → 원래 명령어 재실행
```

예시 문제

Q. 48-bit 가상 주소 공간, 4KB 페이지, 8-byte PTE인 시스템에서 단일 페이지 테이블의 크기는?

```
풀이:
  가상 페이지 수 = 2^48 / 2^12 = 2^36
  단일 PT 크기  = 2^36 × 8 bytes = 2^39 bytes = 512 GB
  → 비현실적 → Multi-Level PT 필요
  Core i7: 4단계 (VPN = 9+9+9+9 bits)
```

Ch7. Dynamic Memory Allocation

블록 구조 (Implicit Free List)

```
┌──────────────────────────────────┐
│  Header (4 bytes)                │
│  [ size (30bits) | 00 | alloc ]  │
│  하위 비트 = alloc bit            │
├──────────────────────────────────┤
│  Payload                         │
│  (application data)              │
├──────────────────────────────────┤
│  Padding (optional)              │
└──────────────────────────────────┘
```

```
alloc=1: 할당된 블록
alloc=0: 가용 블록
size 추출: header & ~0x7
alloc 추출: header & 0x1
```

Segregated Free List 동작

```
크기 클래스별 가용 리스트:
  [1-8]   → 리스트 1 → [8] → [8] → NULL
  [9-16]  → 리스트 2 → [16] → NULL
  [17-32] → 리스트 3 → [32] → [24] → NULL
  [33~∞]  → 리스트 4 → [64] → [48] → NULL

malloc(20) 요청:
  1. [17-32] 클래스에서 first-fit 탐색 → 32 블록 발견
  2. 32 → 20(할당) + 12(새 가용 블록)으로 split
  3. 12는 [9-16] 클래스에 삽입
```

예시 문제

Q. `malloc(100)` 후 `free(p)` 만 하고 병합(coalescing)을 하지 않으면 어떤 문제가 발생하는가?

```
A. False Fragmentation 발생.
   예: [free:50][alloc:100][free:50] → free 후
       [free:50][free:100][free:50]
   병합 없으면 각각 50, 100, 50으로 유지.
   malloc(120) 요청 시 → 합계 200으로 충분하지만
   연속 블록 최대 100 → 할당 실패!
   → Immediate Coalescing 또는 Deferred Coalescing 필요
```

Ch8. Exceptions

예외 처리 흐름

```
User Code 실행 중 이벤트 발생
         │
         ▼
  CPU → Exception Table[k] 조회
         │
         ▼
  커널 Exception Handler 실행
         │
     ┌───┴──────────────────────┐
     │ Interrupt / Trap         │ → I_next로 복귀
     │ Fault (복구 가능)         │ → I_current 재실행
     │ Fault (복구 불가) / Abort│ → 프로세스 종료
```

```
    └──────────────────────────┘
```

System Call 흐름 (open 예시)

```
User: open("file.txt", O_RDONLY)
  → libc: %eax = 2 (open syscall 번호)
  → libc: syscall 명령어 실행
  → Trap 예외 발생 → 커널 모드 전환
  → 커널: syscall_table[2] = sys_open() 호출
  → 파일 열기 수행
  → 반환값 → %rax
  → 유저 모드 복귀
```

Ch9. Processes

fork() + execve() 패턴 (쉘 구현)

```
쉘 main loop:
  while(1):
    read command
    if (builtin) handle locally
    else:
      pid = fork()           ← 자식 생성
      if (pid == 0):
        execve(cmd, args)    ← 자식에서 새 프로그램 실행
      else:
        if (foreground):
          waitpid(pid, ...)  ← 부모가 자식 종료 대기
        else:
          print pid          ← 백그라운드: 대기 없이 계속
```

Process Graph 분석 방법

```
fork() 후 가능한 출력 순서 분석:

코드:
  printf("L0"); fork(); printf("L1"); fork(); printf("Bye");

프로세스 트리:
         [L0]
          │ fork()
    ┌─────┴─────┐
  parent      child1
  [L1]         [L1]
    │ fork()    │ fork()
  ┌─┴─┐       ┌─┴─┐
 P   C2       C3  C4
[Bye][Bye]   [Bye][Bye]

총 출력: L0×1, L1×2, Bye×4
가능한 순서: L0이 항상 첫 번째, 나머지는 비결정적
불가능한 순서: L1이 L0보다 앞에 오는 경우
```

Ch10. Signals

시그널 상태 비트벡터

```
커널이 각 프로세스마다 유지:
```

```
pending  비트벡터: [ 0 0 1 0 0 0 1 0 ... ]
                        ↑       ↑
                    SIGSEGV    SIGCHLD pending

blocked  비트벡터: [ 0 0 0 0 0 0 1 0 ... ]
                              ↑
                          SIGCHLD blocked

pnb = pending & ~blocked
    = 실제로 처리할 시그널 집합

처리 시: 가장 낮은 번호의 bit부터 처리
```

SIGCHLD 핸들러 패턴

```c
void sigchld_handler(int sig) {
    int status;
    pid_t pid;
    // WNOHANG: 이미 종료된 자식만 처리, 블로킹 없음
    // pid=-1: 임의의 자식 프로세스
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        // zombie 회수 완료
    }
}
// 반드시 while 루프 사용!
// if 사용 시: 동시에 여러 자식 종료 → 1개만 처리
// (시그널 큐 없음 → 나머지 zombie 누적)
```

Ch11. Threads

스레드 메모리 공유 모델

```
프로세스 가상 주소 공간:

높은 주소
┌──────────────────────┐
│  Kernel              │ ← 공유
├──────────────────────┤
│  Stack (Thread 2)    │ ← Thread 2 전용 (보호 없음!)
│  Stack (Thread 1)    │ ← Thread 1 전용 (보호 없음!)
├──────────────────────┤
│  Shared Libraries    │ ← 공유
├──────────────────────┤
│  Heap                │ ← 공유 ★ 동기화 필요
├──────────────────────┤
│  .bss / .data        │ ← 공유 (전역변수, static)
│  .text               │ ← 공유 (코드)
└──────────────────────┘
낮은 주소

★ 스택은 VM으로 보호 안됨 → 포인터로 타 스레드 스택 접근 가능
```

Ch12. Synchronization

cnt++ Race 분석

```
cnt++ 어셈블리 분해:
```

```
  L: movq cnt(%rip), %rdx    // %rdx = cnt
  U: addq $1, %rdx           // %rdx++
  S: movq %rdx, cnt(%rip)    // cnt = %rdx

잘못된 인터리빙 (cnt=0에서 시작):
Thread1: L1(rdx1=0)  U1(rdx1=1)            S1(cnt=1)
Thread2:             L2(rdx2=0)  U2(rdx2=1)           S2(cnt=1)
결과: cnt=1  (예상: cnt=2)  → Race Condition!

Progress Graph에서 Unsafe Region:
  Thread1의 L1-U1-S1 구간과 Thread2의 L2-U2-S2 구간이 겹치는 영역
  → Mutex로 해당 구간을 Critical Section으로 보호
```

Producer-Consumer (n-element buffer)

```
구조:
  sbuf_t {
    int *buf;              // 원형 배열
    int n, front, rear;    // 크기, 인덱스
    pthread_mutex_t mutex; // 버퍼 접근 보호
    sem_t empty_slots;     // 초기값 = n (빈 슬롯 수)
    sem_t full_slots;      // 초기값 = 0 (찬 슬롯 수)
  }

sbuf_insert(item):
  wait(empty_slots)   // 빈 슬롯 생길 때까지 대기
  lock(mutex)         // 버퍼 잠금
  buf[++rear % n] = item
  unlock(mutex)       // 버퍼 해제
  post(full_slots)    // 소비자에게 아이템 있음 알림

sbuf_remove():
  wait(full_slots)    // 아이템 생길 때까지 대기
  lock(mutex)
  item = buf[++front % n]
  unlock(mutex)
  post(empty_slots)   // 생산자에게 빈 슬롯 알림
  return item
```

Deadlock 예시 및 회피

```
Deadlock 발생 (락 순서 불일치):
  Thread 0: wait(s0) → wait(s1) → cnt++ → post(s0) → post(s1)
  Thread 1: wait(s1) → wait(s0) → cnt++ → post(s1) → post(s0)

  가능한 상황:
  T0: wait(s0) 성공(s0=0)
  T1: wait(s1) 성공(s1=0)
  T0: wait(s1) → s1=0 → 블로킹!
  T1: wait(s0) → s0=0 → 블로킹!
  → 영원히 대기 = Deadlock

회피 (락 순서 통일):
  Thread 0: wait(s0) → wait(s1) ...
  Thread 1: wait(s0) → wait(s1) ...  ← 동일 순서
  → Circular Wait 조건 제거 → Deadlock 불가
```

Ch13. System-Level I/O

커널 파일 표현 구조

```
프로세스 A            공유 (커널)
Descriptor Table     Open File Table      v-node Table
┌────┬───────┐      ┌────────────────┐   ┌────────────────┐
│ fd0│ stdin │──────→│ pos=0          │──→│ type: CHR      │
│ fd1│stdout │──────→│ refcnt=1       │   │ size: ∞        │
│ fd2│stderr │──────→│                │   └────────────────┘
│ fd3│       │──────→├────────────────┤   ┌────────────────┐
└────┴───────┘       │ pos=1024       │──→│ type: REG      │
                     │ refcnt=1       │   │ size: 4096     │
프로세스 B            └────────────────┘   └────────────────┘
Descriptor Table
┌────┬───────┐
│ fd0│       │──────→ (fork 후 동일 Open File Entry 공유)
│ fd1│       │──────→  refcnt가 2로 증가
└────┴───────┘
```

I/O 리다이렉션 구현

```
$ ls > foo.txt  구현:

1. fork() → 자식 프로세스 생성
2. [자식] fd3 = open("foo.txt", O_WRONLY|O_CREAT|O_TRUNC)
3. [자식] dup2(fd3, 1)   → fd1(stdout)이 foo.txt를 가리킴
                           이전 fd1(terminal) refcnt 감소
4. [자식] close(fd3)     → fd3 불필요, 닫음
5. [자식] execve("ls", ...) → ls의 stdout(fd=1)이 foo.txt에 쓰임

strace 확인:
  openat(..., "foo.txt", O_WRONLY|O_CREAT|O_TRUNC) = 3
  dup2(3, 1) = 1
  close(3) = 0
  execve("/usr/bin/ls", ...)
```

Ch14. Debugger & ptrace

GDB 기반 Debugger 동작 흐름

```
GDB (Tracer)                    Target (Tracee)
──────────────────────────────────────────────

fork()
  └──────────────────────────→ [자식 생성]
                                PTRACE_TRACEME
                                execve("target")
                                → SIGTRAP 자동 발생 → 정지
waitpid() ←── SIGTRAP ────────
PTRACE_PEEKTEXT (코드 읽기)
PTRACE_POKETEXT (0xcc 삽입)   ← 브레이크포인트 설정
PTRACE_CONT ──────────────────→ [실행 재개]
                                0xcc 실행 → SIGTRAP → 정지
waitpid() ←── SIGTRAP ────────
PTRACE_GETREGS (레지스터 읽기)
  → %rip, %rsp 등 출력
PTRACE_POKETEXT (원래 바이트 복원)
PTRACE_SINGLESTEP ────────────→ [명령어 1개 실행 → 정지]
waitpid() ←── SIGTRAP ────────
... 반복
```

Software Breakpoint 동작

```
브레이크포인트 설정:
  원래 바이트 저장: orig = PTRACE_PEEKTEXT(pid, addr)
  0xcc 삽입:       PTRACE_POKETEXT(pid, addr, 0xcc)

실행 중 0xcc 만남:
  CPU → SIGTRAP 예외 → GDB가 수신

브레이크포인트 해제 (재실행 전):
  PTRACE_POKETEXT(pid, addr, orig)  // 원래 바이트 복원
  PTRACE_SETREGS: %rip를 addr로 설정 // 해당 명령어부터 재실행
  PTRACE_SINGLESTEP                  // 1개 실행
  다시 0xcc 삽입                     // 브레이크포인트 복원
```

Ch15. Security Attacks

Cold Boot Attack 흐름

```
신뢰 경계 (OS 신뢰 시):

  CPU Registers  ┐
  CPU Cache      ┤ Trusted
  ──────────────┤
  DRAM           ┤ ← 암호화 키가 일시적으로 존재!
  Flash/HDD      ┘ Untrusted (암호화된 데이터)

공격:
  DRAM을 -50°C로 냉각
  → DIMM 슬롯에서 분리 (1분 내 0.2% 미만 손실)
  → 다른 머신에 삽입
  → DRAM 내용 덤프 → 암호화 키 추출

대응: TRESOR
  AES 키 → CPU 레지스터(DR0~DR3)에만 유지
  DRAM으로 절대 내려가지 않음
```

Meltdown 공격 메커니즘

```c
// 단순화된 Meltdown PoC
char user_array[255 * 4096];

// Step 1: user_array 캐시 전체 flush
for (int i = 0; i < 255; i++)
    clflush(&user_array[i * 4096]);

// Step 2: Transient Execution (투기적 실행)
// 커널 주소 접근 → 권한 위반이지만 즉시 예외 안 남
char secret = *(char *)0xffffffff81a000e0;  // kernel addr
// secret 값을 인덱스로 user_array 접근 → 해당 캐시 라인 로드
maccess(user_array[secret * 4096]);
// 예외 처리: secret 접근은 커밋 안됨 → 레지스터 롤백
// BUT: 캐시 상태는 롤백 안됨!

// Step 3: Flush+Reload로 secret 복원
for (int i = 0; i < 255; i++) {
    t = get_access_time(&user_array[i * 4096]);
    if (t < threshold)  // cache hit!
        recovered_secret = i;  // secret == i
```

```
        }
```

Rowhammer 공격 구조

```
DRAM 물리 구조:
  Bank 0:  Row 0 | Row 1 | Row 2 | Row 3 | ...
           ─────   ─────   ─────   ─────
  aggressor rows: Row 0, Row 2 (반복 접근)
  victim row:     Row 1 (비트 플립 발생)

공격 코드:
  while (1):
    access(row_A)    // DRAM에 직접 접근
    access(row_B)    // (Row 1 인접한 양쪽)
    clflush(row_A)   // 캐시 bypass (L1/L2/L3 우회)
    clflush(row_B)   // → 매번 DRAM 직접 접근 강제

Linux 권한 상승:
  PTE 비트 플립 → PPN 변경 → 다른 물리 페이지 접근
  RW bit(1bit) 플립 확률: 1/64 ≈ 2%
  PPN(20bit) 플립 확률:  20/64 ≈ 31%
```